LangSmith Crash Course | Observability in GenAI

1. Introduction

- **LangSmith** is a platform for developing, debugging, evaluating, and monitoring **LLM/GenAI applications**.
- It works especially well with **LangChain and LangGraph**.
- The main purpose is to understand what is happening inside an AI application.

Simple idea:

AI Application → LangSmith → Observe, Debug, Test, Evaluate

2. Why LangSmith?

When an AI application gives a wrong or unexpected answer, it can be difficult to know **why**.
For example:
User → RAG → Retriever → LLM → Answer
If the answer is incorrect, the problem could be:

- Wrong retrieved documents
- Poor prompt
- Incorrect LLM response
- Unexpected input
- Slow operation
- High token usage

LangSmith helps you inspect these steps instead of treating the AI application as a **black box**.

3. Example 2

- A practical example is used to demonstrate how LangSmith helps analyze an LLM application.
- Instead of only seeing the final answer, you can inspect the individual operations that produced it.
- This makes debugging much easier.

Without LangSmith:

Input → ??? → ??? → Answer

With LangSmith:

Input → Prompt → LLM → Output

You can inspect what happened at each stage.

4. Observability

Observability means being able to understand the internal behavior of an application by looking at its execution data.

For GenAI applications, observability can include:

- Inputs and outputs
- Prompts
- LLM calls
- Retrieved documents
- Execution time
- Token usage
- Errors
- Individual workflow steps

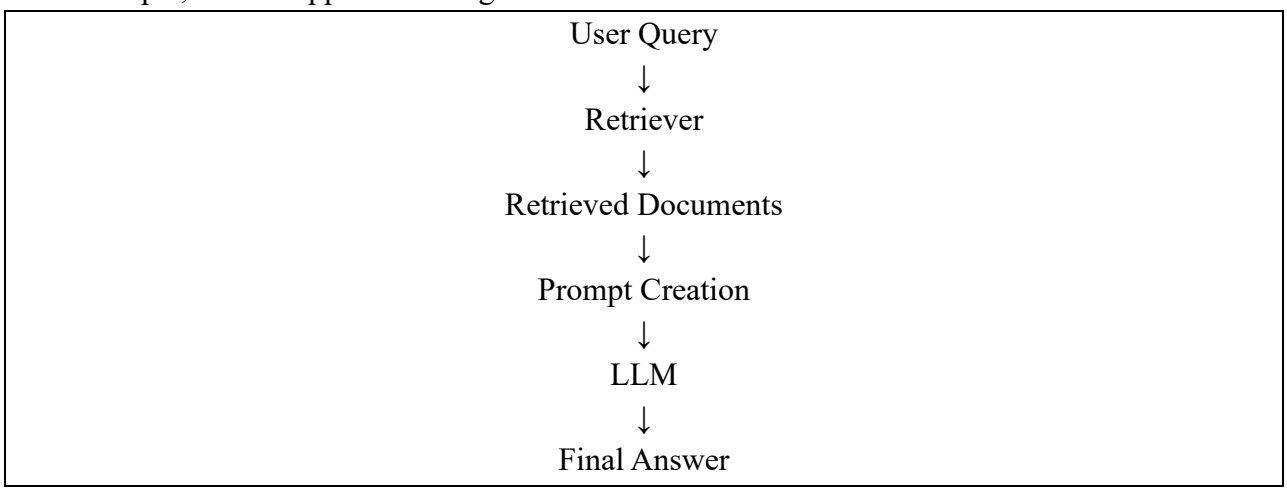
The goal is to answer:

"What exactly happened inside my AI application?"

5. What does LangSmith Trace?

A **trace** represents the execution of an application.

For example, a RAG application might look like:



LangSmith can record these operations so you can inspect them individually.

This allows you to find exactly where something went wrong.

6. Setting up Agenda

The tutorial sets up the learning plan for LangSmith.

The major areas covered are:

- LangSmith basics
- Tracing
- Observability
- Core concepts
- RAG application tracing
- Query testing
- LangGraph integration
- LangSmith features

7. Code Demo

- A basic AI application is connected with LangSmith.
- The application sends execution information to LangSmith.
- After running the application, its execution can be inspected through the LangSmith interface.

Basic flow:

Python Application → LangSmith → Trace

8. Core Concepts

Important LangSmith concepts include:

- **Trace** — The complete execution of an application.
- **Run/Span** — An individual operation inside that execution.
- **Project** — A place where related traces/runs are organized.
- **Metadata** — Additional information attached to executions.
- **Tags** — Labels used to organize and filter traces.
- **Feedback/Evaluation** — Information used to judge the quality of outputs.

The important idea is to break a complex AI execution into smaller observable operations.

9. Code Demo

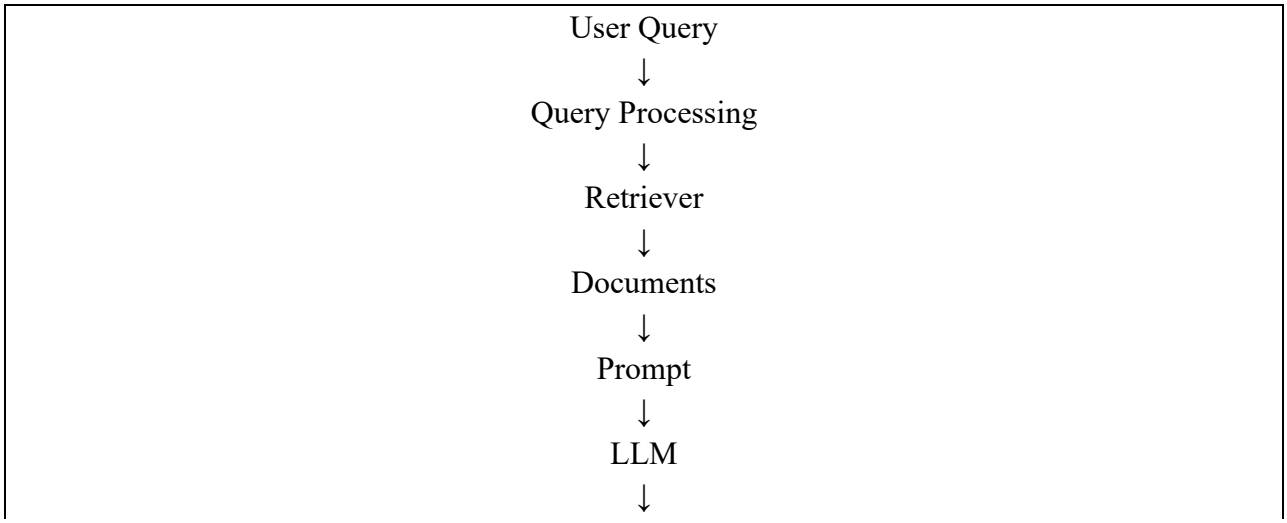
- The concepts are implemented practically.
- The application is run and its execution is sent to LangSmith.
- You can then inspect the execution and understand the internal flow.

This demonstrates how observability works in an actual GenAI application rather than just theoretically.

10. Tracing RAG App

LangSmith becomes particularly useful for **RAG applications**.

A RAG workflow might be:



Answer

LangSmith allows you to trace these individual steps.
For example, if the final answer is incorrect, you can check:
Were the correct documents retrieved? → Was the prompt correct? → Did the LLM generate the expected answer?
This makes debugging RAG systems much easier.

11. Testing Query

- LangSmith can be used to test queries against your AI application.
- You can compare the application's responses and evaluate their quality.
- This is important because an AI application may work correctly for one query but fail for another.

Example:
Query 1 → Good answer
Query 2 → Good answer
Query 3 → Incorrect answer
Query 4 → Poor retrieval
Testing helps identify these weaknesses systematically.

12. LangGraph + LangSmith Integration
LangSmith integrates naturally with **LangGraph**.
LangGraph manages the workflow:

Node A → Node B → Node C → Node D

LangSmith observes the workflow:

LangGraph → LangSmith → Trace each step

This is especially useful for complex agentic workflows because you can see:

- Which node executed
- What state was passed
- What the LLM produced
- Which path the graph followed
- Where an error occurred
- How long each operation took

Simple distinction:

- **LangGraph** → Builds and controls the AI workflow.
- **LangSmith** → Observes, debugs, tests, and evaluates the workflow.

13. Features of LangSmith
Important LangSmith capabilities include:

- **Tracing** — See how an AI application executes.
- **Debugging** — Find problems inside workflows.

- **Evaluation** — Measure response quality.
- **Testing** — Test applications with different inputs.
- **Monitoring** — Observe applications in production.
- **RAG debugging** — Inspect retrieval and generation.
- **LangGraph integration** — Trace graph-based workflows.
- **Feedback** — Collect information about output quality.
- **Performance analysis** — Understand latency and execution behavior.

14. Outro

The tutorial concludes the introduction to LangSmith and its role in GenAI development.

Key Takeaway

LangSmith is the observability and evaluation layer for GenAI applications.

Think of the relationship like this:

LangGraph
↓
Builds the AI workflow
↓
LangSmith
↓
Observes + Traces + Tests + Evaluates + Debugs

In simple terms:

LangGraph tells the AI application what to do; LangSmith helps you see, understand, test, and improve what it did.